AI, ML, DL, LLMs, and AI Security Research Map

I would organize this topic into six layers: **foundations**, **model architecture**, **training lifecycle**, **security threats**, **defensive standards**, and **operational monitoring**. That structure makes the field easier to remember than a flat glossary, because it shows how the ideas connect. AI is the umbrella, ML is a subset of AI, DL is a subset of ML, and LLMs are deep-learning language models that are trained on large corpora and use context to generate or transform text.

1) Foundations: the core terms

AI (Artificial Intelligence). The broad category of systems that perform tasks associated with human intelligence. ISO and Google both describe AI as a broad field that includes many techniques and disciplines.

ML (Machine Learning). A subset of AI where systems learn from data and improve without being explicitly programmed. Google's AI/ML material consistently frames ML as data-driven learning using algorithms.

DL (Deep Learning). A subset of ML that uses artificial neural networks with many layers to learn complex patterns. Google describes deep learning as ML with multi-layer neural networks.

ML algorithm. The method used to learn patterns from data and produce predictions or decisions. In practice, the algorithm is the "learning procedure," while the model is the learned result.

LLM (Large Language Model). A statistical language model trained on massive datasets to generate, translate, summarize, and answer questions. Google Cloud describes LLMs as large deep neural networks trained on huge amounts of data.

LLaMA. Meta's family of foundation language models. Meta describes Llama as openly available foundation models trained on very large token corpora, with newer generations expanding capability and context length.

RLHF (Reinforcement Learning from Human Feedback). A fine-tuning method where human preference data is used to steer a model toward better behavior. OpenAI's published work on instruction-following language models explicitly uses RLHF to align outputs with human feedback.

Pre-training. The large initial training phase where a model learns general structure from broad data before task-specific tuning. Google and Meta both describe LLMs as pre-trained on massive data before later specialization.

Contextual understanding. This is the model's ability to use surrounding tokens and prior text to shape the next output. Transformers and long-context systems are built around that idea.

2) Neural networks and transformers

A **neural network** is a layered computational model. The **input layer** receives raw data, **hidden layers** transform features, and the **output layer** produces the prediction. Google's docs describe that exact structure.

A **transformer neural network** is a model architecture built around attention rather than recurrence or convolution. The original Transformer paper states that the architecture is based solely on attention mechanisms. That is why transformers became the backbone of modern LLMs.

3) The four main ML learning categories

Supervised learning. Trains on labeled examples, where the input and the correct answer are both known. Google's glossary and supervised-learning page describe this as learning from labeled data to predict outcomes.

Unsupervised learning. Trains on unlabeled data to find structure, clusters, or hidden patterns. Google's glossary distinguishes unlabeled examples from labeled ones and uses that distinction in the supervised/unsupervised split.

Semi-supervised learning. Uses a small amount of labeled data and a larger amount of unlabeled data. Google's glossary places it in the family of methods that use unlabeled examples during training.

Reinforcement learning. An agent learns by interacting with an environment and receiving rewards or penalties. Google describes RL as trial-and-error learning with feedback from the environment.

4) The machine learning lifecycle

The lifecycle is best understood as an iterative loop, not a straight line. Google's ML lifecycle and MLOps material emphasizes production pipelines for data processing, training, serving, monitoring, and retraining.

A practical lifecycle looks like this:

1. **Define the problem.** Decide what the model should predict or automate.
2. **Collect data.** Gather training, validation, and test data.
3. **Clean data.** Remove errors, duplicates, noise, and inconsistent labels. This is essential before useful training happens.
4. **Feature engineering.** Turn raw data into useful model inputs. Google explicitly separates this as a major phase in ML development.

5. **Select algorithm and train the model.** Choose the method that fits the problem and learn from the data.
6. **Evaluate.** Test whether the model actually works on unseen data.
7. **Deploy.** Put the model into production so it can serve predictions or responses.
8. **Monitor.** Watch for drift, skew, bad performance, or abuse, then retrain as needed.

What “scalable ML” means

Scalable ML means the model and its pipeline can grow with data, traffic, and complexity without breaking operations. Google’s Vertex AI material explicitly supports serverless training, reserved clusters, and scaling with Ray, while MLOps is described as the discipline that manages the ML lifecycle efficiently at scale.

5) AI-specific vulnerabilities and attack classes

The cleanest way to study AI attacks is to split them into **input attacks**, **training/supply-chain attacks**, **runtime/output attacks**, and **agentic attacks**. OWASP’s LLM and AI-agent guidance, plus MITRE ATLAS, are the most useful starting points.

Input attacks

Prompt injection and **jailbreaking** try to override instructions or make the model behave in unintended ways. OWASP describes this as manipulation through crafted inputs, including direct and indirect prompt injection.

Training and supply-chain attacks

Data poisoning manipulates pre-training, fine-tuning, or embedding data so the model learns backdoors, bias, or malicious behavior. OWASP treats this as a major risk, and MITRE ATLAS is designed to map adversarial tactics against AI-enabled systems.

Model theft / model extraction is unauthorized copying or extraction of model behavior, weights, or capabilities. OWASP describes it as exfiltration of the model itself, and notes that attackers may build a shadow model by querying the API.

Runtime and output attacks

Sensitive information disclosure / data leakage happens when the model or its application reveals private or proprietary data. OWASP treats this as a top risk, especially when outputs are used without adequate filtering.

Insecure output handling means the application trusts model output too much and fails to validate it before use. OWASP flags this because model output can trigger downstream security issues.

Model denial of service / unbounded consumption is resource exhaustion caused by heavy prompts, repeated inference, or expensive loops. OWASP now treats this as a major LLM risk.

Model drift is not always an attack, but it is a serious operational risk: performance degrades as real-world data changes. Google's model-monitoring docs define drift and skew detection as core production monitoring controls.

Agentic attacks

When AI can use tools, memory, or actions, the attack surface expands. OWASP's agent guidance highlights **tool abuse**, **privilege escalation**, **data exfiltration**, **memory poisoning**, **goal hijacking**, and **excessive autonomy** as key risks.

6) Enhanced AI attacks used by attackers

NIST's Generative AI risk profile and Google's threat intelligence both say attackers can use AI to scale or improve **phishing**, **vishing**, **malware development**, **social engineering**, and **deepfakes**. That does not make the model itself "malware"; it means AI raises attacker productivity and realism.

A practical way to remember it:

- **Phishing / vishing / spear phishing** becomes more convincing.
- **Deepfakes** can impersonate people and defeat trust checks.
- **Malware writing and adaptation** become faster and cheaper.
- **Information operations** scale better because content generation becomes trivial.

7) Compact IBM Cost of Data Breach 2025 takeaways

IBM's 2025 report says the **global average breach cost was USD 4.44 million**, down 9% year over year, and that **97%** of organizations with AI-related breaches lacked proper AI access controls. It also says **63%** lacked AI governance policies, **shadow AI** added about **USD 670,000** to breach costs in the cases studied, and extensive use of AI/security automation reduced breach costs by about **USD 1.9 million**.

The most useful interpretation for you is simple: **AI adoption without governance is now a measurable security cost**, not just a theoretical risk. IBM also highlights identity security, data security, AI oversight, security automation, and resilience as the main action areas.

8) AI safety and securing AI systems

A strong AI security posture has five layers:

1. Secure the data. Control training data, embeddings, prompts, outputs, and model artifacts. IBM recommends discovery, classification, access control, encryption, and key management.

2. Secure the model. Protect weights, APIs, fine-tuning pipelines, and deployment endpoints from theft, poisoning, and misuse. OWASP's model-theft and poisoning guidance maps directly to this.

3. Secure the application layer. Filter inputs, validate outputs, sandbox tools, and limit agent permissions. OWASP's prompt-injection and agent-security guidance is the practical reference here.

4. Monitor continuously. Watch for skew, drift, abnormal consumption, and malicious behavior. Google's model-monitoring docs and MLOps guidance make monitoring a core production control.

5. Red-team the system. Use adversarial testing and threat models that are AI-specific, not just ordinary appsec. MITRE ATLAS exists for exactly that purpose.

9) Standards and frameworks you should know

NIST AI RMF 1.0. A voluntary framework for managing AI trustworthiness and risk across the lifecycle.

ISO/IEC 42001:2023. The first global AI management system standard. ISO says it helps organizations govern AI risks, transparency, ethics, and continuous improvement.

ISO/IEC 23894:2023. Guidance on AI risk management.

ISO/IEC 22989:2022. AI concepts and terminology. This is the vocabulary standard that helps everyone use the same language.

ISO/IEC 23053:2022. A framework for AI systems using machine learning. ISO describes it as a conceptual framework and shared terminology for ML-based AI systems.

ISO/IEC 27090. This is under development and is specifically about cybersecurity guidance for AI threats and compromises. It is not yet a final mature standard, so treat it as emerging guidance rather than settled doctrine.

ISO/IEC 27091. Also under development, focused on privacy risks in AI systems and ML models.

NIST control overlays for AI systems. NIST's AI security overlay work is aimed at adapting security controls to AI-specific components like training data, model weights, and configuration. That makes it useful for implementation, not just policy.

OWASP Top 10 for LLMs and OWASP Agentic AI guidance. These are practical application-layer references for prompt injection, data poisoning, model theft, output handling, and agent misuse.

10) Operational terms you listed

DoH port. DNS over HTTPS uses **TCP 443** by default, because it is carried over normal HTTPS traffic.

Windows ephemeral/dynamic port range. Microsoft's current default dynamic client port range is **49152–65535**.

SYN flood timeout. There is not one universal setting with that exact name; Linux and Windows expose TCP retry, backlog, and syncookie-related controls instead. Linux kernel docs explicitly mention syncookies to prevent SYN flood attacks, and Red Hat documents the "possible SYN flooding" log line that appears when the backlog is under pressure.

SSH login log / SSH attempt. OpenSSH normally logs the beginning, authentication, and termination of each connection, and Ubuntu man pages point to system logging such as `auth.log` and journald-related tools.

11) The best mental model for you

If you are learning this as a cybersecurity person, remember this chain:

AI → ML → DL → LLM → agentic AI,
then think separately about **how it is built**, **how it is attacked**, **how it is defended**, and **how it is governed**.

If you want, I can turn this into a **GitHub-ready markdown document** with headings, a glossary table, and a cleaner section for PDF export.